It's worth briefly mentioning that Ruby does have traditional for loops, and you could start the code with something like for i in 0...5. But the for loop is too knowledgeable about the array; it magically knows that we're iterating over five elements, and it retrieves values in turn from the array. To do this, it has to know that the structure it is working with is an array of two-element subarrays. All that knowledge makes the code brittle—subject to breaking if the underlying data changes. The enumeration construction is more robust and flexible.

However you use them, enumeration and code blocks are among the more interesting features of Ruby, so let's spend a while looking into them.

Blocks

A *block* is a chunk of code either enclosed between braces or between the keywords do and end. The two forms are identical except for precedence, which is rarely an issue in practice. Ruby style favors using braces for blocks that fit on one line and do/end when a block spans multiple lines. Also, style has spaces between the brace and the code to distinguish a block from a Hash literal.

```
some_array.each { |value| puts value * 3 }
```

```
sum = 0
other_array.each do |value|
  sum += value
  puts value / sum
end
```

You can think of a block as being somewhat like the body of an anonymous method. Like a method, the block can take parameters (but, unlike a method, those parameters appear at the start of the block between vertical bars). Both the blocks in the preceding example take a single parameter, value. And, like a method, the body of a block is not executed when Ruby first sees it. Instead, the block is saved away to be called later.

Blocks can appear in Ruby source code only immediately after the *invocation* of a method. If the method takes parameters, the block appears after these parameters. You can think of the block as being an extra parameter passed to that method. Let's look at a simple example that sums the squares of the numbers in an array:

```
sum = 0
[1, 2, 3, 4].each do |value|
  square = value * value
  sum += square
end
puts sum
```

produces:

```
30
```

The block is being called by the each method once for each element in the array, with each element passed to the block as the value parameter in turn. But there's something else going on. Take a look at the sum variable. It's declared outside the block, updated inside the block, and then passed to puts after the each method returns.

This example illustrates an important rule: the block has access to the variable scope outside the block, and does not, by default, create new variables with existing names. There's only